



Università
di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e
Informatica

Advanced Programming Languages

C++ overview

Docente:

Marco Grassia

marco.grassia@unict.it

A.A. 2025/2026

C++

- **C++** is a high-performance, multi-paradigm programming language designed for **system-level programming, application development, and performance-critical software**
- Developed by **Bjarne Stroustrup** at **Bell Labs** in the early 1980s
- Originally named “*C with Classes*”, renamed **C++** in 1983
- **Designed** in the industry **for industrial use**, not academic experimentation
- Combines the **efficiency and control of C** with **object-oriented features**

“I have always wished for my computer to be as easy to use as my telephone; my wish has come true because I can no longer figure out how to use my telephone.”

– Bjarne Stroustrup

(developer of the C++ language)



C++ : key characteristics

- **Compiled: the compiler**, a separate program, is **used to turn C++ source code into executable code**, a form directly executed by the CPU
- **Strongly statically typed and unsafe: conversions between variable types must be made by the programmer** (strong typing) **but can be circumvented** when needed (**unsafe**)
- **Multi-paradigm: it supports many programming styles** (object oriented, procedural, functional, generic, concurrent) which translates into powerful abstractions (e.g., objects, templates, lambdas, operator overloading, ...)

Multi-paradigm?

- “Multiparadigm programming is a **fancy way of saying “programming using more than one programming style**, each to its best effect.”
- *For example, using object-oriented programming when run-time resolution between different object types is required and generic programming when static type safety and run-time performance is at a premium. [...]*
- *I find the most compelling cases for multiparadigm programming are found where **techniques from different paradigms are used in close collaboration to write code that is more elegant and more maintainable** than would be possible within a single paradigm”*

Key characteristics of C++

- **C compatible:** call C libraries directly and C code is nearly 100% valid C++ code
- **No automatic memory management:** there is **no garbage collector**; memory management is manual or via smart pointers. The programmer is in control of memory usage
- **Modern C++ introduces new powerful features** to help the programmer handle memory, concurrency, performance and safety

Key characteristics of C++

- **Availability**

- Call C libraries directly and C code is (nearly!!) 100% valid C++ code
- C++ compilers are available for virtually all platforms and architectures.
- Rich ecosystem of libraries and tools (Boost, STL, Qt, etc.)

- **Portability:** C++ source code is relatively portable across platforms

- Supported by a wide range of compilers (GCC, Clang, MSVC, Intel, etc.)
- Used in cross-platform development (e.g., Qt, Unreal Engine)

Key characteristics of C++

- **Expressiveness:**
 - supports **multiple paradigms**
 - Enables concise and **powerful abstractions** (e.g., templates, lambdas, operator overloading)
 - Allows fine-grained control over syntax and semantics
- **Compactness:**
 - Code can be highly optimized and minimal, especially in performance-critical contexts
 - Features like inline functions, templates, and RAII reduce boilerplate*

*: **Boilerplate** code is code that is **necessary but repetitive**, often **required to set up** a framework or **environment before the actual logic can be implemented**. It's usually not unique to the specific application but is needed for things **like configuration, imports, or structure**.

C with Classes?

Why even start from C?

- C++ was designed as an extension of the C language, and choosing C as a foundation brought several strategic advantages
- **Efficiency**
 - C provides **direct access to memory and hardware**
 - **Minimal runtime overhead**, ideal for performance-critical applications
 - C++ inherits this efficiency, making it suitable for systems programming
- **Flexibility**
 - C is a **general-purpose** language with a **simple** and **powerful syntax**
 - C++ builds on this by **adding support for multiple paradigms** (OOP, generic, functional)

C with Classes?

Why even start from C?

- **Availability**

- C **compilers** are **available on** virtually **every platform** and architecture
- This ensures that C++ inherits a wide deployment base and tooling ecosystem

- **Portability**

- C code is *relatively* **portable across platforms**
- C++ maintains this portability while offering more abstraction and expressiveness

- **Legacy Integration**

- **Vast amounts of** existing **code and libraries** are written in C
- C++ can **interface directly with C code**, enabling gradual **migration** and **reuse**

Why not all C code is C++ code

- **Additional keywords** in C++: identifiers valid in C **may become reserved** in C++ (e.g., *new*, *class*, *template*)
- **C++ has stricter type checking:**
 - Implicit conversions are more restricted
 - Assigning a *void** to another pointer type requires an explicit cast (in C it doesn't)
- **String literals and constants:**
 - In C++, **string literals are *const char[]***
 - **In C they are not**, which breaks code that modifies them
- Standard library differences: different headers (<cstdio> vs <stdio.h>)

*“C makes it easy to shoot yourself in the foot;
C++ makes it harder, but
when you do it blows your
whole leg off.”*

– Bjarne Stroustrup (1986)

(here with John Romero, founder of id Software and
lead designer of Doom)



Similarities Between C and C++

- **Code Structure:** both languages share a similar overall structure.
- **Compilation process:** the compilation workflow is largely the same
- **Basic Syntax:** most operators and keywords from C are also present in C++ with the same functionality; C++ slightly extends the grammar, but the core grammar remains the same
- **Memory Model:**
 - Both languages have a low-level memory model close to the hardware.
 - Concepts like **stack**, **heap**, and **static variables** exist in both

Distinctive Features of C++

- **Input and Output Methods:** C uses printf/scanf, while C++ introduces *cin/cout* and stream-based I/O
- **Programming Paradigm:** C is procedural; C++ supports **object-oriented programming** and **generic programming**
- **Standard Template Library (STL):** provides containers, algorithms, and iterators for generic programming
- **Namespaces:** prevents name collisions by grouping identifiers
- **Exception Handling:** introduces try, catch, and throw for robust error management
- **File Extensions:** C source files: .c; C++ source files: .cpp

Choose C++ when

- **Program performance matters:** dealing with large amounts of data, multiple CPUs, complex algorithms, etc.
- **Programmer productivity is less important:** it is faster to produce working code in Python, R, Matlab or other scripting languages!
- The **programming language itself can help organize your code**
- **Access to libraries**, examples include: Nvidia's CUDA Thrust library for GPUs; Boost; Qt; PyTorch; TensorFlow; Eigen; POCO; Gtest; ...

“If you’re **not at all** interested in performance, shouldn’t you be in the **Python room** down the hall?”
— Scott Meyers

(author of [Effective Modern C++](#))



Notable software written in C/C++

- **Embedded Systems, IoT, and Robotics**
 - C++ is widely used in **embedded software** for IoT devices, microcontrollers, and real-time systems
 - In **real-time embedded systems**, where deterministic and immediate responses are critical, C++ can meet strict timing requirements with careful design
- **Simulation and Scientific Computing**
 - **Simulation and Modeling:** Flight simulators, traffic systems, and complex system modeling often rely on C++ for performance.
 - **Scientific and High-Performance Computing:** Physics, engineering, bioinformatics, and computational chemistry use C++ for intensive calculations

Notable software written in C/C++

- **Industrial and Financial Applications**
 - **Industrial Control Systems:** automation and precision control in manufacturing environments
 - **Financial Applications:** high-frequency trading and risk analysis require efficient data processing and complex calculations, making C++ a preferred choice
- **Libraries, Frameworks, and AI**
 - Many core **libraries and frameworks** (e.g., STL) are written in C++ and serve as building blocks for diverse applications.
 - **Artificial Intelligence and Machine Learning:** C++ is used for performance-critical AI components, often integrated with specialized libraries

Notable software written in C/C++

- **Operating Systems and System Software**
 - C++ powers parts of **operating systems**, including kernels and device drivers, thanks to its direct hardware control and efficient memory management
- **Gaming, Graphics, and Multimedia**
 - **Game Development:** High-performance games and graphics engines use C++ with APIs like DirectX and OpenGL.
 - **Graphics and Animation:** Software for 3D modeling, rendering, and visual effects often relies on C++ for speed and control.
 - **Desktop Applications:** Productivity tools, office suites, and multimedia editors frequently use C++ for responsiveness and portability.

Notable software written in C/C++

- Operating Systems: **Windows, MacOS, Linux**
- Browsers: **Google Chrome / Chromium, Mozilla Firefox, Opera Safari** (WebKit)
- Graphics and Multimedia: **Adobe Photoshop, Illustrator, Premiere, Autodesk Maya, AutoCAD, 3ds Max**
- Office suites: **Microsoft Office, OpenOffice**
- Game and Render engines: **Unreal Engine, Unity, CryEngine, Source Engine** (Valve), **Blender**
- Interpreters and engines: **JavaScript (V8, SpiderMonkey), Python (CPython), Lua, R, MATLAB, Perl, PHP**
- Databases: **MySQL, PostgreSQL, Oracle DB, MongoDB**

C++ influences

Language	Influence on C++
Simula	Introduced classes and object-oriented programming concepts. C++ inherited class-based encapsulation and inheritance.
Algol 68	Inspired operator overloading , reference types , and flexible variable declarations (e.g., declaring variables anywhere in a block).
BCPL	Source of the // single-line comment syntax , later adopted by C++ and other languages.
Ada	Influenced templates through its generic programming constructs . Also contributed to ideas of strong typing and modularity .
CLU	Inspired parameterized modules , which influenced C++ templates. Also contributed to exception handling design.
ML	Influenced exception handling mechanisms in C++. ML's functional style also indirectly shaped later features like lambdas and type inference.

History

- **1979: Bjarne Stroustrup** begins work **at Bell Labs** (New Jersey, USA) on enhancing the C language with object-oriented features
- **1983:** The language is **officially named C++**, a play on the increment operator ++, symbolizing an evolution of C
- **Late 1980s:** C++ **gains popularity** in industry for its combination of **low-level efficiency** and **high-level abstraction**
- **1990:** First **commercial release** by AT&T; the language becomes widely adopted in systems and application development
- **1998: C++98 standard** is published **by ISO**, formalizing the language and its standard library

C++ standards

- **C++03**: Minor revision (2003). Focused on **bug fixes** and **clarifications**
- **C++11**: Published in 2011, it represents **a turning point**
 - **auto keyword** for type inference
 - **Lambda expressions** for inline functions
 - **Move semantics** for efficient resource transfer
 - **Smart pointers** (*unique_ptr*, *shared_ptr*)
 - **Range-based for loops**, *nullptr*, *enum class*
- **C++14**: Published in 2014. Incremental improvements:
 - **Return type deduction**
 - **Generic lambdas**
 - **Binary literals** (0b1010)
 - Minor STL enhancements

C++ standards

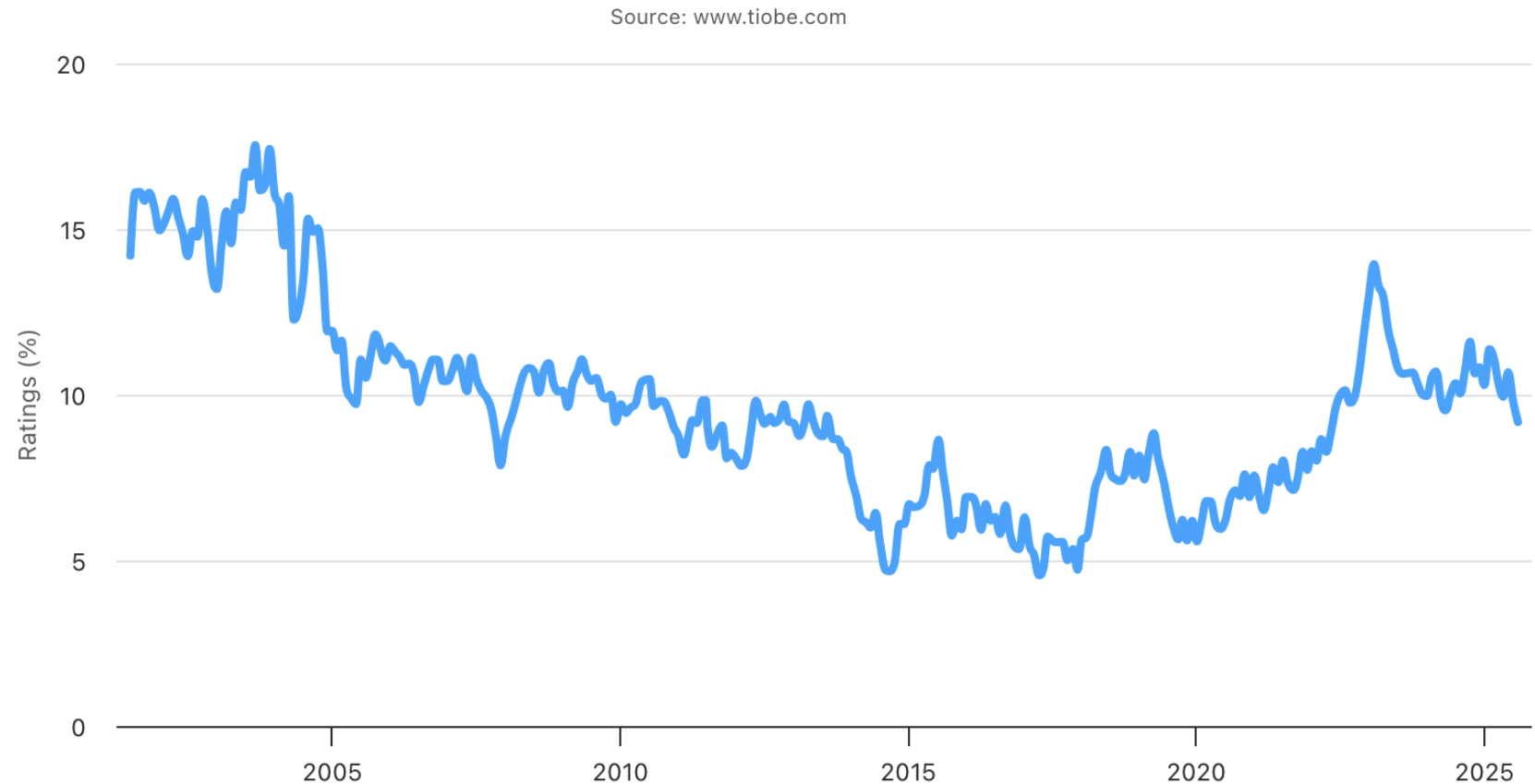
- **C++17.** Published in 2017. Focused on usability and performance:
 - **Structured bindings** (auto [x, y] = pair;).
 - **Inline variables**, if and switch with initializers
 - **Filesystem library**, std::optional, std::variant
- **C++20.** Major modernization (2020). Introduced:
 - **Concepts** for template constraints
 - **Ranges library** for clean iteration
 - **Coroutines** for asynchronous programming
 - **Modules** for better compilation and encapsulation
 - **Calendar and time zones**, constexpr enhancements
- **C++23.** Latest version (2024). Focused on refinement and usability:
 - **Improved standard library** features
 - **Expanded support for constexpr**
 - **Better Unicode handling**, std::expected, and more

C++ diffusion: StackOverflow's survey

- Pretty stable in the last 5 years:
 - 2020: 23,9%
 - 2021: 24,5%
 - 2022: 22,5%
 - 2023: 22,5%
 - 2024: 23%
 - 2025: 23.5%

C++ diffusion: TIOBE index

- Measure of popularity of programming languages calculated from the number of search engine results for queries containing the name of the language.

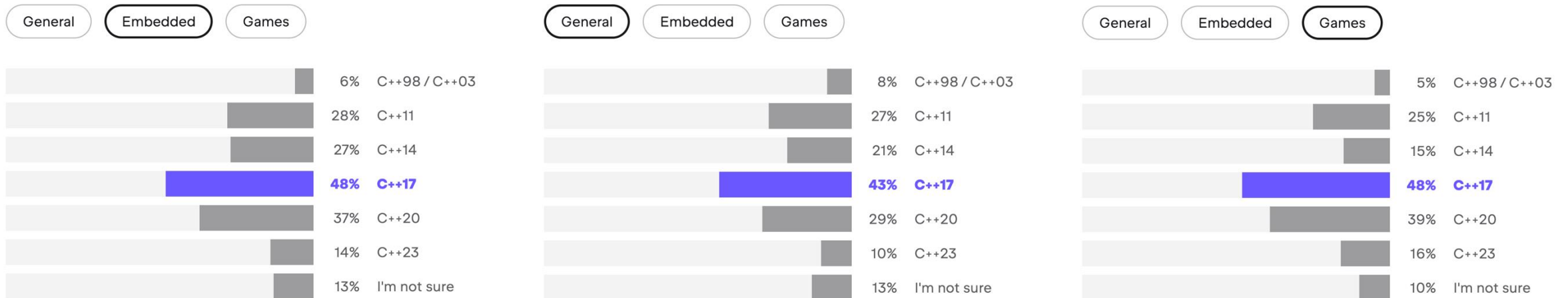


C++ diffusion: JetBrains' Dev. Ecosys. report

	2017	2018	2019	2020	2021	2022	2023	2024	
	65%	64%	69%	70%	69%	65%	61%	61%	JavaScript
	32%	41%	49%	55%	52%	53%	54%	57%	Python
	60%	55%	61%	61%	60%	54%	52%	51%	HTML / CSS
	42%	47%	56%	56%	54%	49%	52%	48%	SQL
	47%	51%	50%	54%	49%	48%	49%	46%	Java
	12%	17%	25%	28%	29%	34%	34%	37%	TypeScript
	–	29%	40%	39%	37%	34%	34%	36%	Shell
	17%	18%	20%	27%	23%	25%	25%	25%	C++
	20%	22%	24%	22%	21%	23%	21%	22%	C#
	15%	16%	17%	23%	19%	20%	19%	18%	C
	8%	12%	18%	19%	17%	19%	17%	18%	Go
	30%	26%	29%	27%	32%	20%	18%	17%	PHP
	2%	9%	16%	17%	14%	16%	15%	14%	Kotlin
	–	2%	5%	7%	6%	9%	10%	11%	Rust

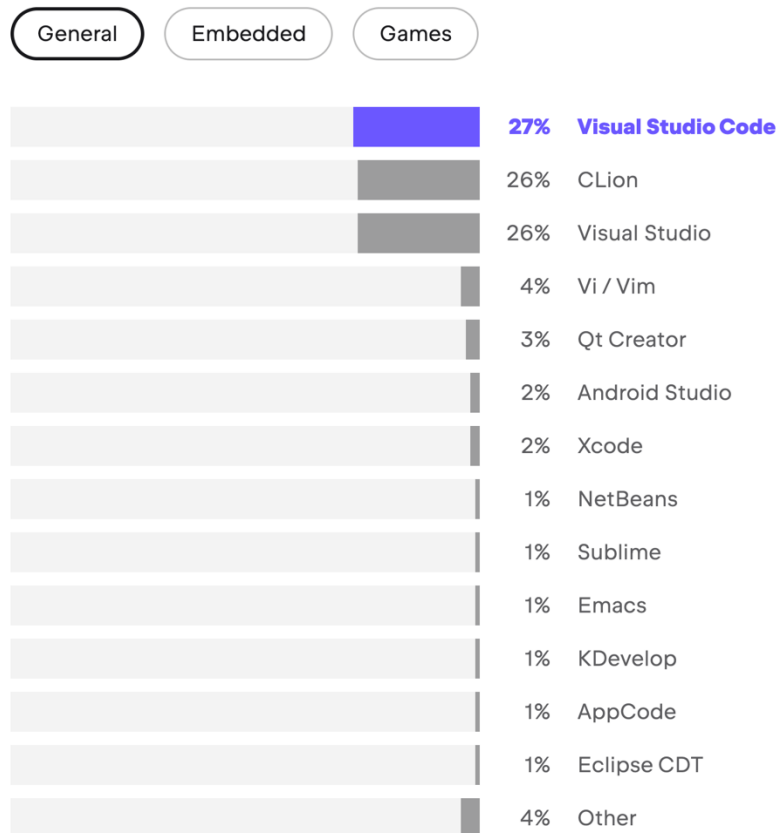
JetBrains' Developer Ecosystem report 2024:

Which C++ standards do you regularly use?

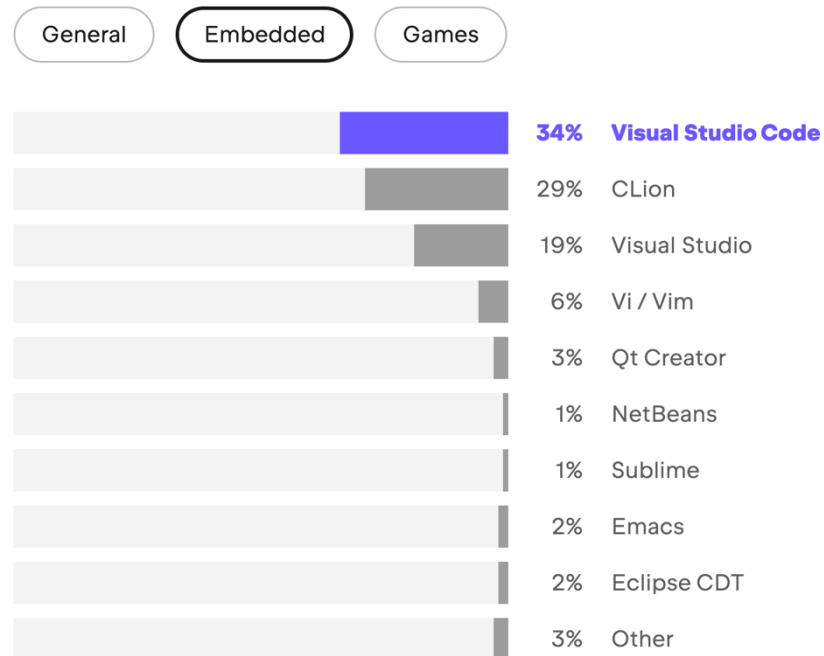


JetBrains' Developer Ecosystem report 2024:

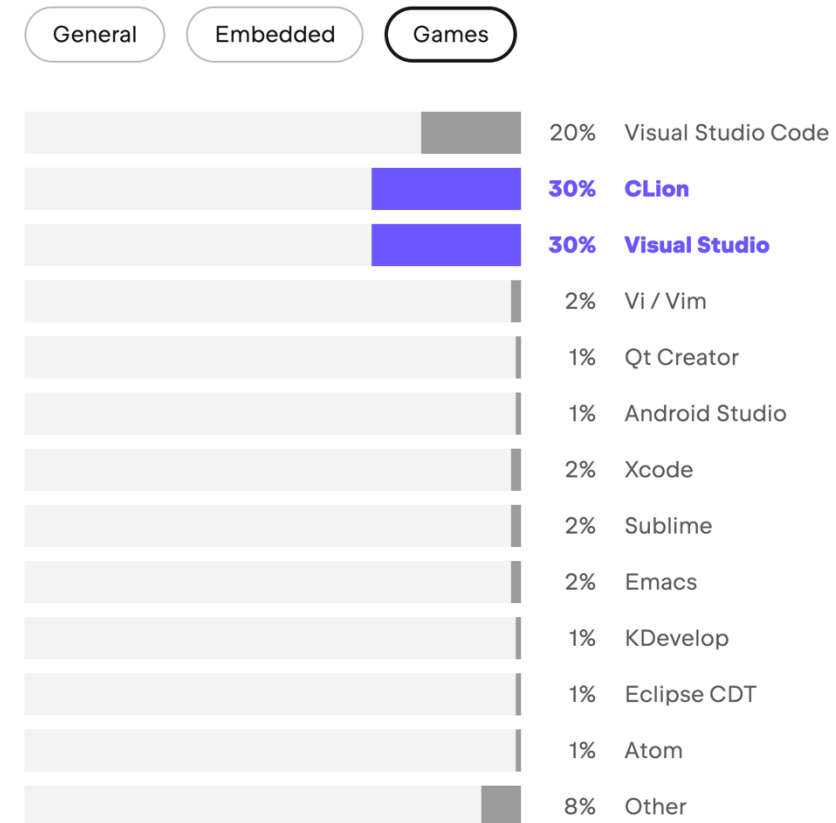
Which IDE or editor do you use the most for C++ development?



A.A. 2025/2026

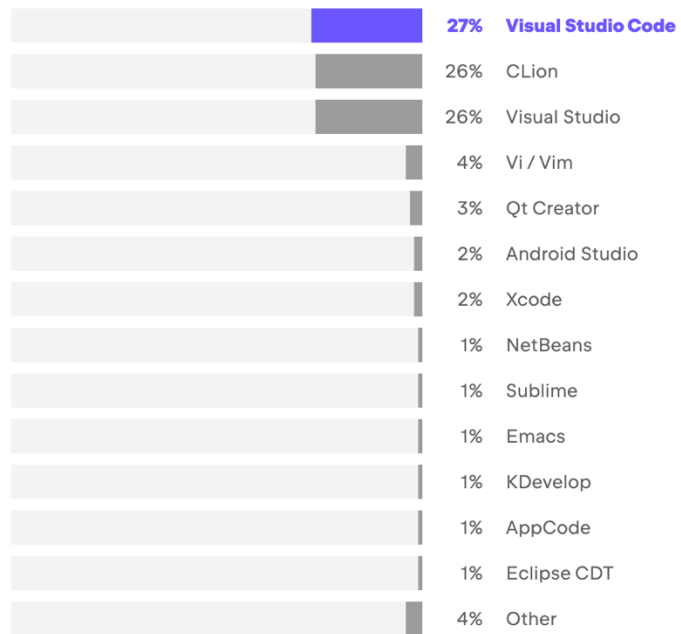


APL: C++



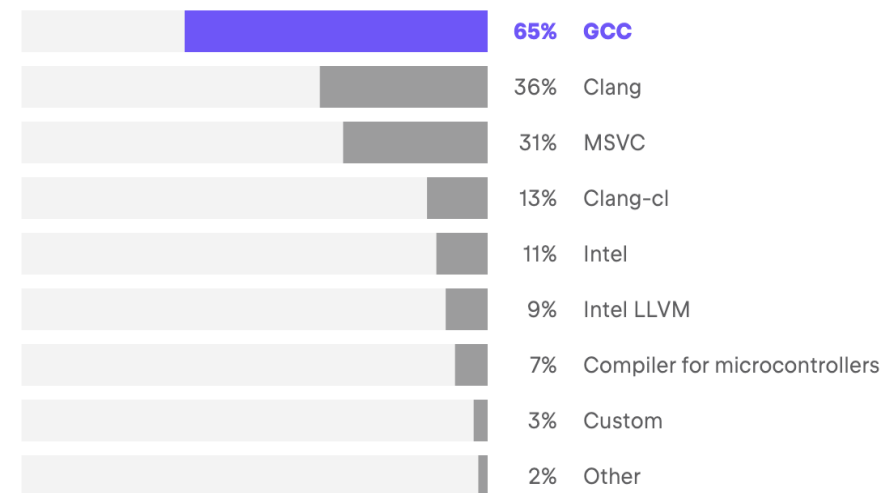
49

Ide

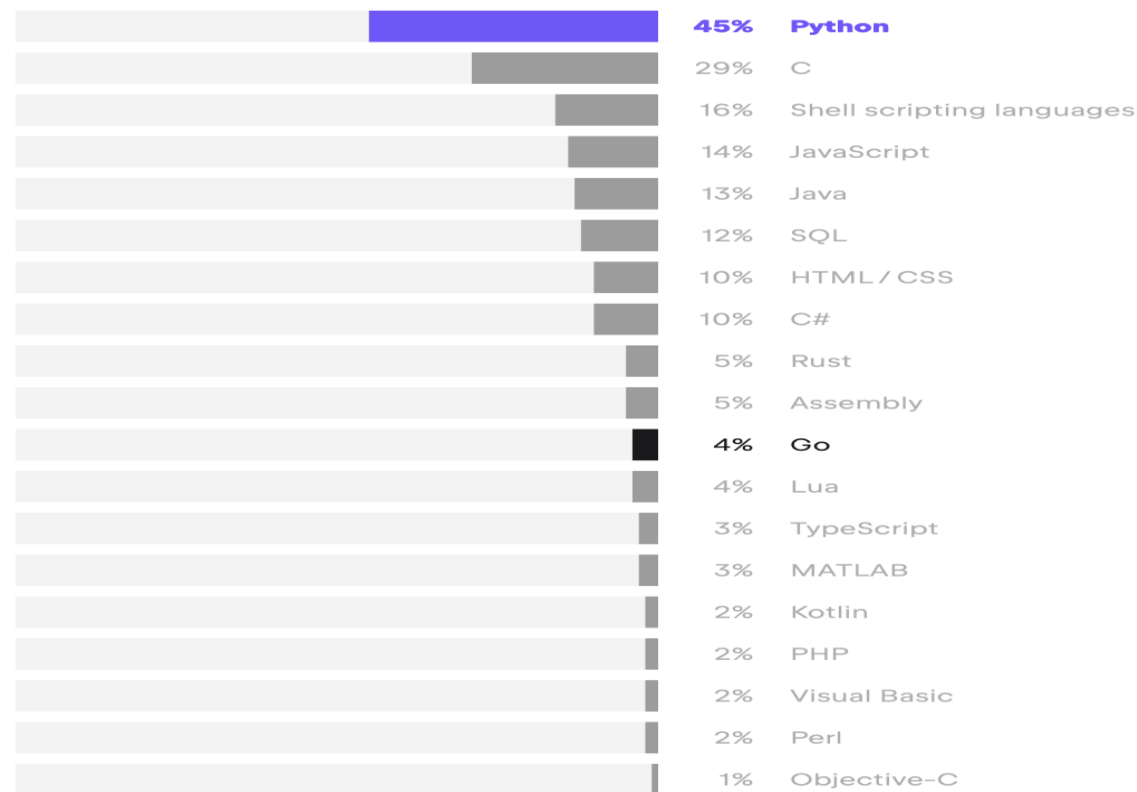


COMPILER

Which compilers do you regularly use?



Which other programming languages do you use in your projects, along with C++?



*“Guido van Rossum wanted a language that was usable by everybody, and I wanted a language that was good for engineers. **I don't want everybody to program the brakes in my car.**”*

– Bjarne Stroustrup

(developer of the C++ language)



Why has C++ remained successful?

- **Despite many competitors, C++ has been popular for over 30 years** and will likely remain so in the foreseeable future
 - **Effective for Complex Problems:** complex programs can be implemented efficiently
 - **Object-Oriented Programming Works:** OOP models real-world problems effectively
 - **Unique Combination:** no other language quite matches C++'s blend of **performance**, **expressiveness**, and ability to handle **complex systems**

Why has C++ remained successful?

Despite many competitors, C++ has been popular for over 30 years and will likely remain so in the foreseeable future

- **Not proprietary:** in 1989, all rights to the language were transferred from AT&T to standardization bodies (first ANSI, then ISO)
- **Continuous evolution:** incorporated new ideas over time (e.g., **templates, exceptions, STL**)
- **Stability:** high degree of backward compatibility across versions
- **General-Purpose Design:** design choices favor general-purpose features rather than application-specific ones

Aspect	C	C++
Origin	Developed by Dennis Ritchie (1969–1973) at AT&T Bell Labs	Developed by Bjarne Stroustrup in 1979 as “C with Classes” at AT&T Bell Labs
Paradigm	Procedural	Multi-paradigm: procedural, object-oriented, generic, functional
OOP Support	✗ No support for encapsulation, inheritance, polymorphism	✓ Full support for object-oriented programming
Relationship	C is (mostly) a subset of C++	C++ is (mostly) a superset of C
Keyword Count	C90: 32 C99: 37 C11: 44 C23: ~59	C++98: 63 C++11: 84 C++20: 93 C++23: 93
Programming Approach	Top-down	Bottom-up
File Extension	.c	.cpp, .cc, .cxx, .c++
Design Focus	Process-oriented	Data-oriented

Aspect	C	C++
Data & Functions	Separated (procedural)	Encapsulated in classes/objects
Information Hiding	Not supported	Supported via encapsulation and access specifiers
Data Types	Built-in types + user-defined (structs, unions)	Built-in + user-defined (classes, templates, etc.)
Memory Management	Manual via malloc, calloc, free	Manual via new, delete; smart pointers (std::unique_ptr, etc.)
Type Safety	Weak(er) type checking	Strong(er) type checking
Function Overloading	✗ Not supported	✓ Supported
Operator Overloading	✗ Not supported	✓ Supported
Functions in Structs	✗ Not allowed	✓ Allowed (member functions)
Access Modifiers in Structs	✗ Not supported	✓ Supported (public, private, protected)

Aspect	C	C++
Namespaces	✗ Not available	✓ Available (namespace)
Standard I/O	stdio.h (printf, scanf)	iostream (std::cout, std::cin)
Reference Variables	✗ Not supported	✓ Supported (T&)
Virtual & Friend Functions	✗ Not supported	✓ Supported
Inheritance	✗ Not supported	✓ Supported
Exception Handling	✗ Not supported	✓ Supported (try, catch, throw)
Meta-programming	Macros, _Generic (C11)	Templates, constexpr, concepts
Type Punning with Unions	✓ Allowed (C99 and later)	⚠ Mostly undefined behavior; use std::bit_cast in C++20+
Named Initializers	✓ Allowed out-of-order (C99+)	⚠ Allowed in C++20 but must follow member order; no inheritance support